Data Science Capstone Project Report

Predicting SpaceX Falcon 9 First-Stage Landing Success

Dataset used for this report: 101 launch records, covering 2010–2020.

Repository: https://github.com/sudharsan0317/spacex-capstone-project

Prepared as a compact slide-style report aligned to the IBM capstone rubric. The supplied CSV is the analysis source; the API and scraping slides document reproducible collection approaches and should be replaced with execution evidence if required by the course.

1. Executive Summary

This project examines whether SpaceX Falcon 9 first-stage landing outcomes can be explored and predicted from launch characteristics. The supplied dataset contains 101 records.

Using a simple label where Landing_Outcome begins with “Success” = 1, 61 records are labeled successful and 40 are labeled not-success. This binary simplification groups several distinct outcomes together.

Exploratory analysis compares launch sites, payload mass, outcomes, and year-by-year activity. A baseline Logistic Regression and Random Forest were tested on one stratified holdout split.

On that split, Logistic Regression achieved about 65.4% accuracy and Random Forest about 61.5%. These are preliminary results, not proof of reliable operational prediction.

Project repository: https://github.com/sudharsan0317/spacex-capstone-project

2. Introduction and Problem Statement

Reusable launch vehicles depend on recovering the first stage safely and consistently. Understanding how launch conditions relate to landing outcomes can support operational planning and risk analysis.

Problem statement: using historical launch records, can available features such as payload mass, launch site, orbit, customer, and booster family help estimate the landing-success label?

The goal is an educational predictive-analysis workflow: collect/inspect data, clean fields, visualize patterns, query summaries with SQL, and compare classification models.

The analysis is limited by the small dataset and by the simplified target definition.

Project repository: https://github.com/sudharsan0317/spacex-capstone-project

3. Data Collection — SpaceX API

Reproducible API workflow (example request)

The report analysis uses the supplied SpaceX launch CSV. The following is a reproducible API collection pattern for the project; it is not a claim that the API was run during this submission.

Recommended workflow: request launch records from the public SpaceX API, inspect the JSON response, select launch date/site/rocket/payload/outcome fields, flatten nested fields, and export a tabular dataset.

Repository with project materials: https://github.com/sudharsan0317/spacex-capstone-project

```
import requests
url = "https://api.spacexdata.com/v4/launches"
response = requests.get(url, timeout=30)
response.raise_for_status()
launches = response.json()
# Next: normalize selected JSON fields into a dataframe
# and save the cleaned result to CSV.
```

Project repository: https://github.com/sudharsan0317/spacex-capstone-project

4. Data Collection — Web Scraping

Documented scraping approach

For launch-site or mission context not available in a structured source, a scraping workflow should identify a permitted public page, inspect its HTML, extract only relevant table rows, normalize labels, and validate the extracted records against the main dataset.

A responsible scraper should check the website's terms and robots guidance, use a descriptive user agent where appropriate, limit request frequency, and cite the source.

This submission's numerical analysis is based on the supplied CSV; no unverified scraped records have been added.

```
# Example pattern only — adapt to the chosen public page
import requests
import pandas as pd
from bs4 import BeautifulSoup

# html = requests.get(page_url, timeout=30).text
# tables = pd.read_html(html)
# launches_table = tables[0]
# Validate columns, dates, duplicates, and missing values
# before joining with the analysis dataset.
```

Project repository: https://github.com/sudharsan0317/spacex-capstone-project

5. Data Wrangling Methodology

Loaded the CSV with pandas and parsed the Date column to derive Year. Converted payload mass to numeric and treated missing numeric values as missing rather than as zero.

Created Landing_Success = 1 when Landing_Outcome starts with “Success”; all other outcomes were labeled 0. This is a modeling simplification, not a statement that all non-success categories are equivalent.

For the baseline models, numeric and categorical missing values were imputed, categorical fields were one-hot encoded, and numeric fields were scaled for Logistic Regression.

Before production use, validate duplicates, category spellings, missingness, and whether “no attempt” outcomes should be excluded or modeled separately.

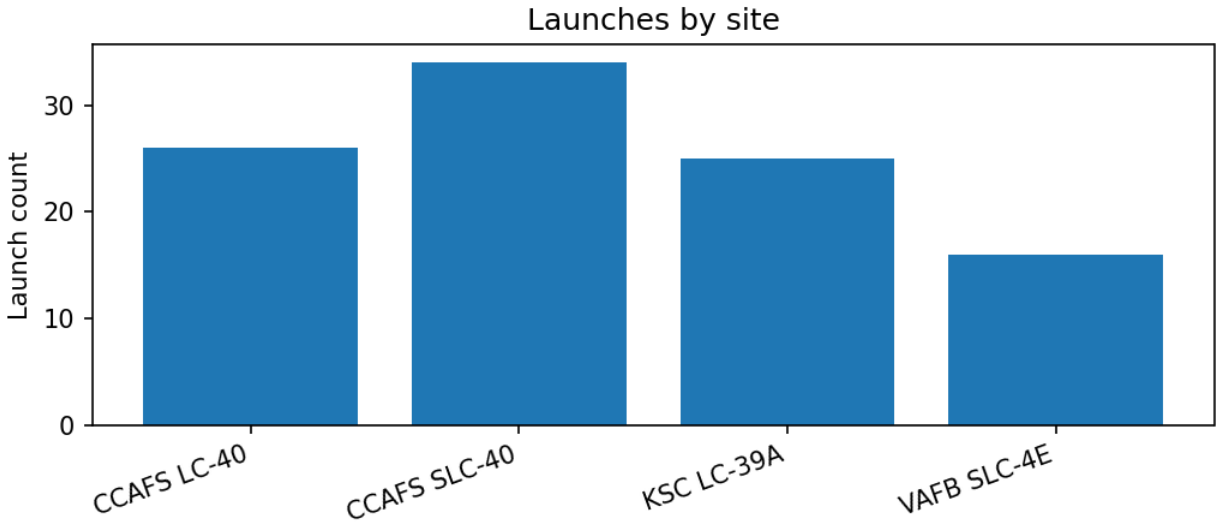
Project repository: https://github.com/sudharsan0317/spacex-capstone-project

6. EDA with Data Visualization

Launch counts by site show how records are distributed across launch locations.

The yearly trend chart compares launch volume and labeled successes over time; yearly counts can vary with the dataset's coverage.

The payload plot helps inspect whether payload mass alone separates the two target labels. It should not be interpreted as causal evidence.

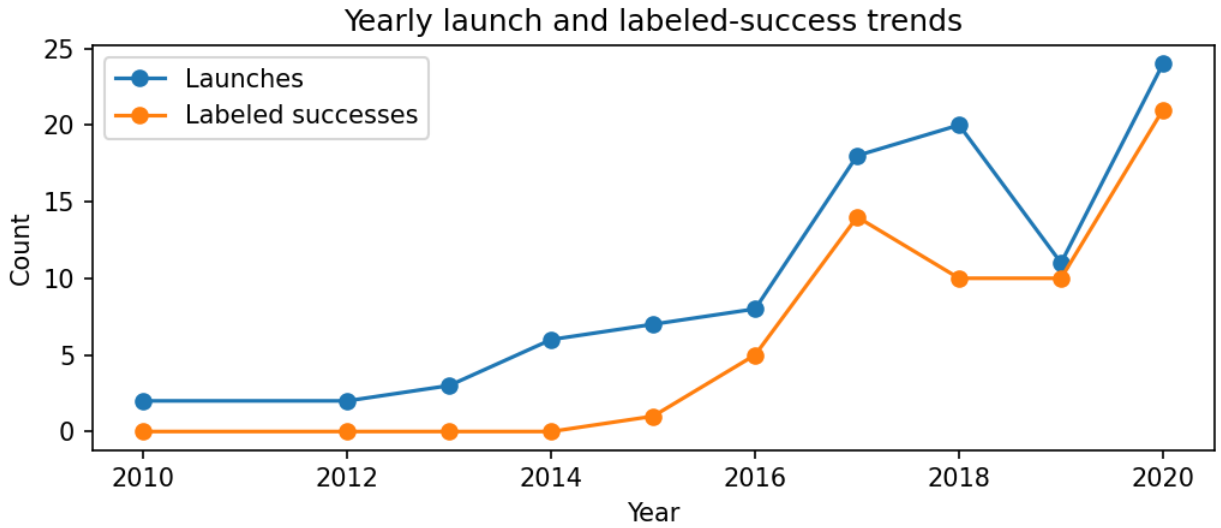


Project repository: https://github.com/sudharsan0317/spacex-capstone-project

7. EDA Visualization Results — Yearly Trends

The line chart summarizes launch counts and labeled-success counts by year.

Use this view to identify changes in activity over time, but avoid treating year-to-year changes as causal explanations without additional variables and a larger dataset.

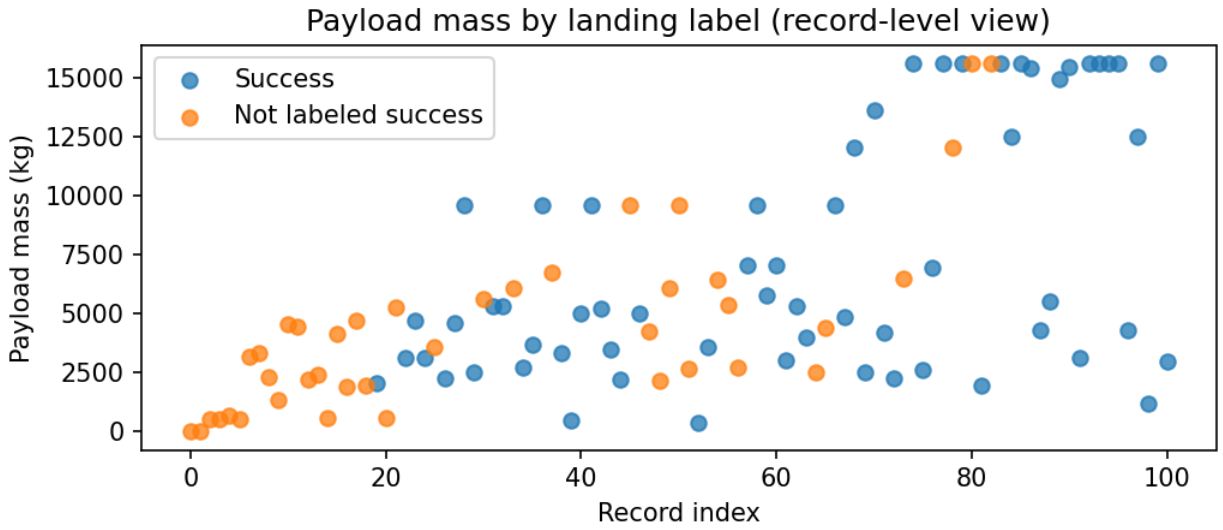


Project repository: https://github.com/sudharsan0317/spacex-capstone-project

8. EDA Visualization Results — Payload vs Outcome

The plot compares payload mass for records labeled successful and not labeled successful.

The target is based on the supplied Landing_Outcome string. Missing payload values are omitted from this chart, and the scatter is descriptive rather than causal.



Project repository: https://github.com/sudharsan0317/spacex-capstone-project

9. EDA with SQL

SQL was used to group launch records by site, count launches, count labeled successes, and calculate descriptive labeled-success percentages.

These results are dataset-level summaries. Differences can reflect the mix of missions, time periods, and booster types at each site; they should not be interpreted as a causal site ranking.

```
SELECT Launch_Site,
COUNT(*) AS launches,
SUM(CASE WHEN Landing_Outcome LIKE 'Success%' THEN 1 ELSE 0 END)
AS labeled_successes,
100.0 * SUM(CASE WHEN Landing_Outcome LIKE 'Success%' THEN 1 ELSE 0 END)
/ COUNT(*) AS labeled_success_pct
FROM spacex_launches
GROUP BY Launch_Site
ORDER BY launches DESC;
```

Project repository: https://github.com/sudharsan0317/spacex-capstone-project

10. SQL Results — Launch Sites

The table reports counts and labeled-success percentages calculated from the supplied dataset.

Launch site	Launches	Labeled successes	Labeled success %
CCAFS SLC-40	34	25	73.5%
CCAFS LC-40	26	6	23.1%
KSC LC-39A	25	20	80.0%
VAFB SLC-4E	16	10	62.5%

Project repository: https://github.com/sudharsan0317/spacex-capstone-project

11. SQL Results — Payload, Ranking, and Time

Recommended SQL extensions for the final analysis include: payload bands and outcome counts; site rankings by labeled-success percentage with a minimum-launch threshold; and monthly/yearly launch counts.

A ranking should always show its denominator. Sites with very few records can have unstable percentages, so compare counts and rates together.

The year trend is shown in the EDA section. Payload-level and time-based SQL queries should be included in the repository so results can be reproduced.

```
-- Payload-band summary (adapt column/table names as needed)
SELECT CASE
WHEN PAYLOAD_MASS__KG_ < 5000 THEN '< 5,000 kg'
WHEN PAYLOAD_MASS__KG_ < 10000 THEN '5,000–9,999 kg'
ELSE '10,000+ kg'
END AS payload_band,
COUNT(*) AS launches
FROM spacex_launches
GROUP BY payload_band
ORDER BY launches DESC;
```

Project repository: https://github.com/sudharsan0317/spacex-capstone-project

12. Interactive Visual Analytics — Folium Map

The project bundle includes a Folium HTML map concept with markers for the four launch-site labels in the dataset. Open the HTML file in a browser to inspect the markers.

The current site labels do not provide precise launch-record coordinates, so marker placement is approximate and should be checked against authoritative site coordinates before drawing proximity conclusions.

The map should show launch-site markers and launch records, with a popup for site name and available summary metrics.

```
Repository: spacex_launch_sites_map.html
Map limitation: approximate site coordinates; validate before interpreting proximity.
```

Project repository: https://github.com/sudharsan0317/spacex-capstone-project

13. Interactive Visual Analytics — Plotly Dash

The project bundle includes a Dash app source file intended to display outcome counts and payload-versus-outcome views.

To run locally: install dash, plotly, and pandas; keep Spacex.csv beside the Python file; then execute the app. The PDF cannot itself provide live dashboard interaction.

For rubric evidence, include a screenshot of the running app and ensure the repository URL points to the source.

```
pip install dash plotly pandas
python spacex_dashboard.py
```

Project repository: https://github.com/sudharsan0317/spacex-capstone-project

14. Predictive Analysis — Models and Evaluation

Two baseline classifiers were evaluated on the same stratified 75/25 holdout split: Logistic Regression and Random Forest.

Features included payload mass, launch site, orbit, customer, and booster family. Preprocessing imputed missing values and one-hot encoded categorical variables.

The test set was small (26 records), so one split can produce unstable estimates. Cross-validation and tuning are recommended before drawing stronger conclusions.

Model	Accuracy	Precision	Recall	F1
Logistic Regression	65.4%	70.6%	75.0%	72.7%
Random Forest	61.5%	66.7%	75.0%	70.6%

Project repository: https://github.com/sudharsan0317/spacex-capstone-project

15. Predictive Analysis — Confusion Matrix, Conclusion & Next Steps

On the recorded holdout split, Logistic Regression was the better of the two tested baselines by accuracy and F1. This is a modest result and does not establish generalization.

Before resubmission, add a clearly labeled confusion matrix for the selected model and explain false positives/false negatives in the context of landing prediction.

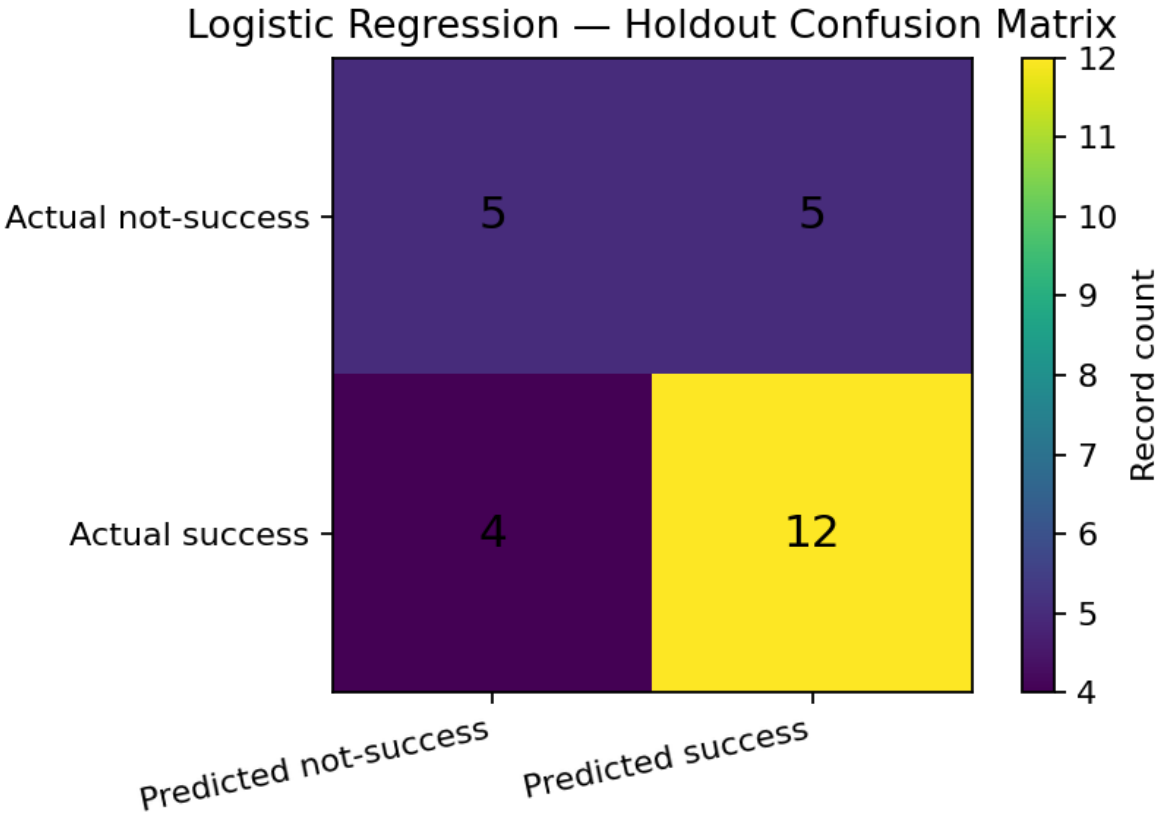
Next steps: use cross-validation, tune model parameters, compare additional models, define a more defensible target for “no attempt” outcomes, and verify the API/scraping workflows with actual execution evidence.

Conclusion: the dataset supports descriptive analysis and a baseline model, but the sample size and target simplification limit confidence in predictive claims.

Project repository: https://github.com/sudharsan0317/spacex-capstone-project

Predictive Analysis — Confusion Matrix

Logistic Regression holdout results. Counts: true negatives = 5, false positives = 5, false negatives = 4, true positives = 12. This 26-record test split is small, so the estimates are uncertain.



Interpretation: 12 successful landings were correctly identified; 4 successful cases were missed; 5 non-success cases were incorrectly predicted as success; and 5 non-success cases were correctly identified.